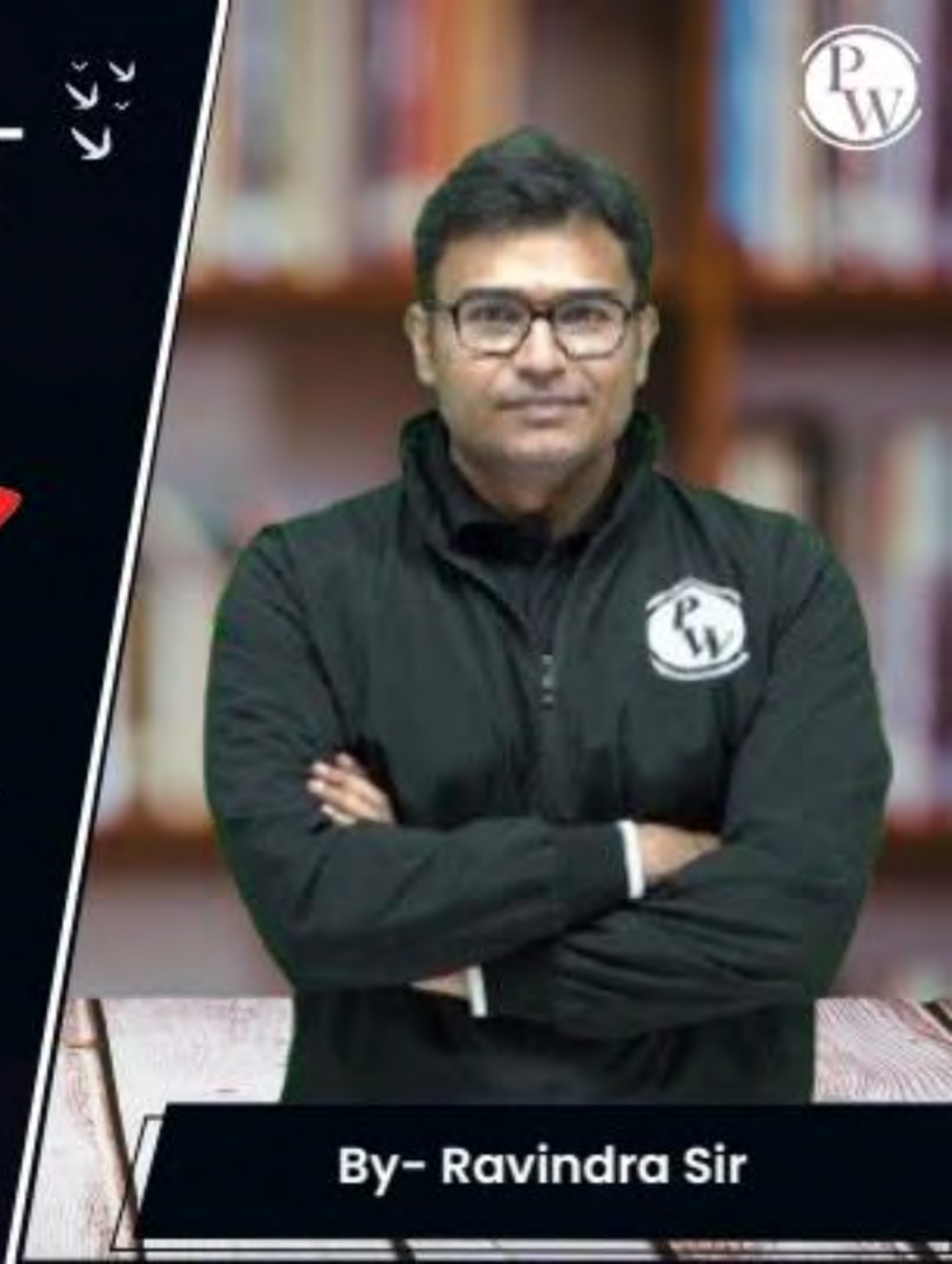
Computer Science & IT

Compiler Design

Parsers

Lecture No. 02



By- Ravindra Sir

Recap of Previous Lecture

Topic

Grammar

Topic

Topic

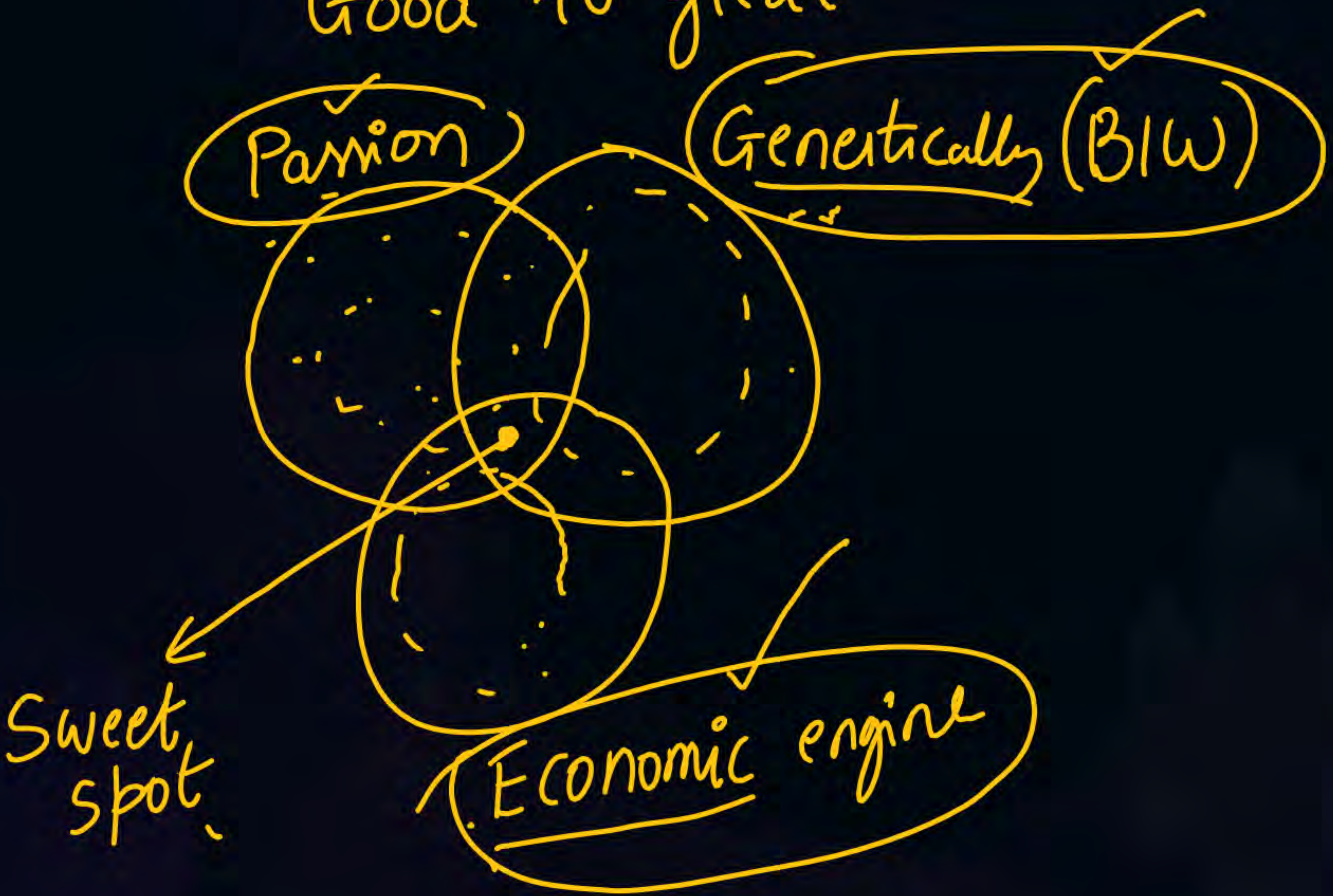
Topics to be Covered

Topic Grammar

Topic Papers

Topic

Good to great



$$R \rightarrow R+R \;/\; R^R \;/\; R^* \;/\; a \;/\; b \;/\; \epsilon$$

$* > \cdot > +$

$\cdot > \cdot$

$+ > +$

Regular expression

$$E \rightarrow E+T \;/\; T$$

$$T \rightarrow T.F \;/\; F$$

$$F \rightarrow F^* \;/\; a \;/\; b \;/\; \epsilon$$

Gate :-

$A \rightarrow A \$ B / B$

$B \rightarrow B \# C / C$

$C \rightarrow C @ D / D$

$D \rightarrow d$

$@ \gtrdot \# \gtrdot \$$ $\quad @ \lessdot @$

$\$ \lessdot \$$

$\# \gtrdot \#$

Gate :-

$E \rightarrow E * F$

$E \rightarrow F + E$

$\quad / F$

$F \rightarrow F - F$

$\quad / id$

same level

ambi

$(* \doteq + \lessdot -)$

$* \gtrdot *$

$+ \lessdot +$

$- \; ? \; -$

Recursion

Left recursion

$A \rightarrow A\alpha / \beta$

TDP LRX

A( ) ∞
{
A( )
α
}

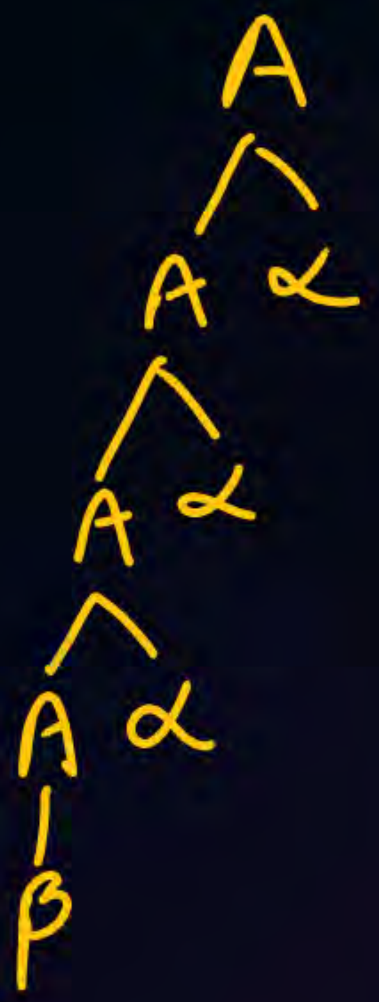
Righ Recusive

$A \rightarrow \alpha A / \beta$

A( )
{
α
A( )
}

Recursion

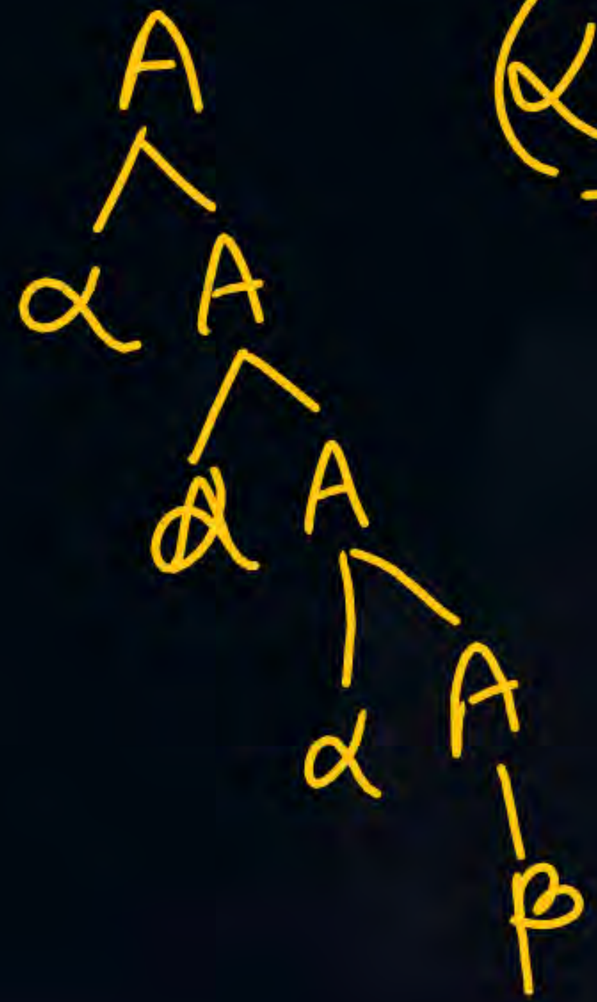
Left recursion		Right Recursive
$A \to A\alpha / \beta$	$\neq$	$A \to \alpha A / \beta$



$(\beta\alpha^*)$

$A \to \beta A'$

$A' \to \alpha A' / \epsilon$



$(\alpha^* \beta)$

$$A \to A\alpha_1 / A\alpha_2 / A\alpha_3 \text{-----} A\alpha_n$$

$$/ \beta_1 / \beta_2 / \beta_3 \text{-----} \beta_n$$

$$A \to \beta_1 A' / \beta_2 A' / \beta_3 A' / \text{------} / \beta_n A'$$

$$A' \to \alpha_1 A' / \alpha_2 A' / \alpha_3 A' \text{---} / \alpha_n A' / \epsilon$$

$$\underset{A}{E} \to \underset{A}{E}\ \underset{\alpha}{+T} \,/\, \underset{\beta}{T} \Rightarrow$$

$$A \to \beta A'$$
$$A' \to \alpha A' / \epsilon$$

$$E \to TE'$$
$$E' \to +TE' / \epsilon$$

$$\underbrace{S}_{A} \rightarrow \underbrace{S}_{A}\underbrace{0S1S}_{\alpha} / \underbrace{01}_{\beta}$$

$$S \rightarrow 01S'$$

$$S' \rightarrow 0S1SS' / \epsilon$$

$S \rightarrow (L) / x$

$$\underset{A}{\underline{L}} \rightarrow \underset{A}{\underline{L}}\underset{\alpha}{\underline{, S}} / \underset{\beta}{\underline{S}}$$

$L \rightarrow S L'$

$L' \rightarrow , S L' / \epsilon$

- Grammar
  - ambiguous
    - X Parser (operator precedence parse)
  - unambiguous

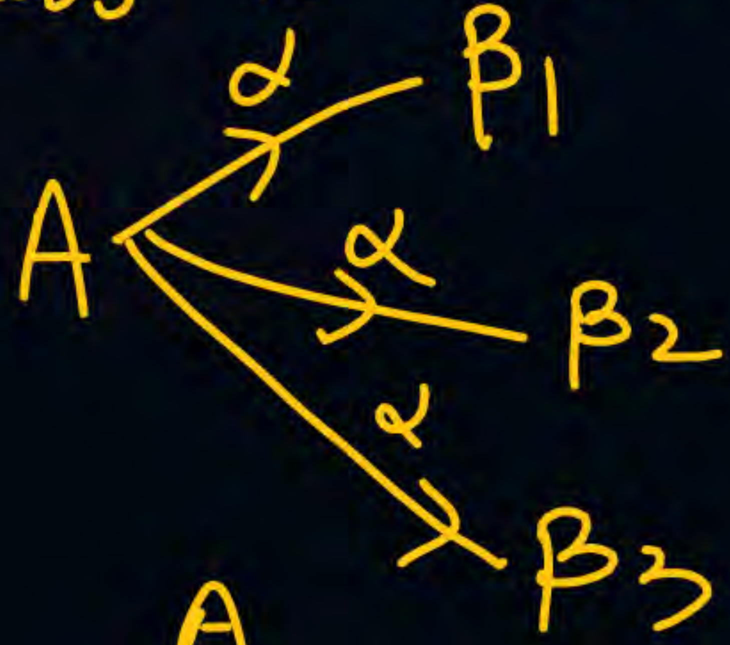
- Grammar
  - LR XTDP
  - RR

- Grammar
  - Deterministic ✓
  - Non Deterministic ✓
    - XTDP

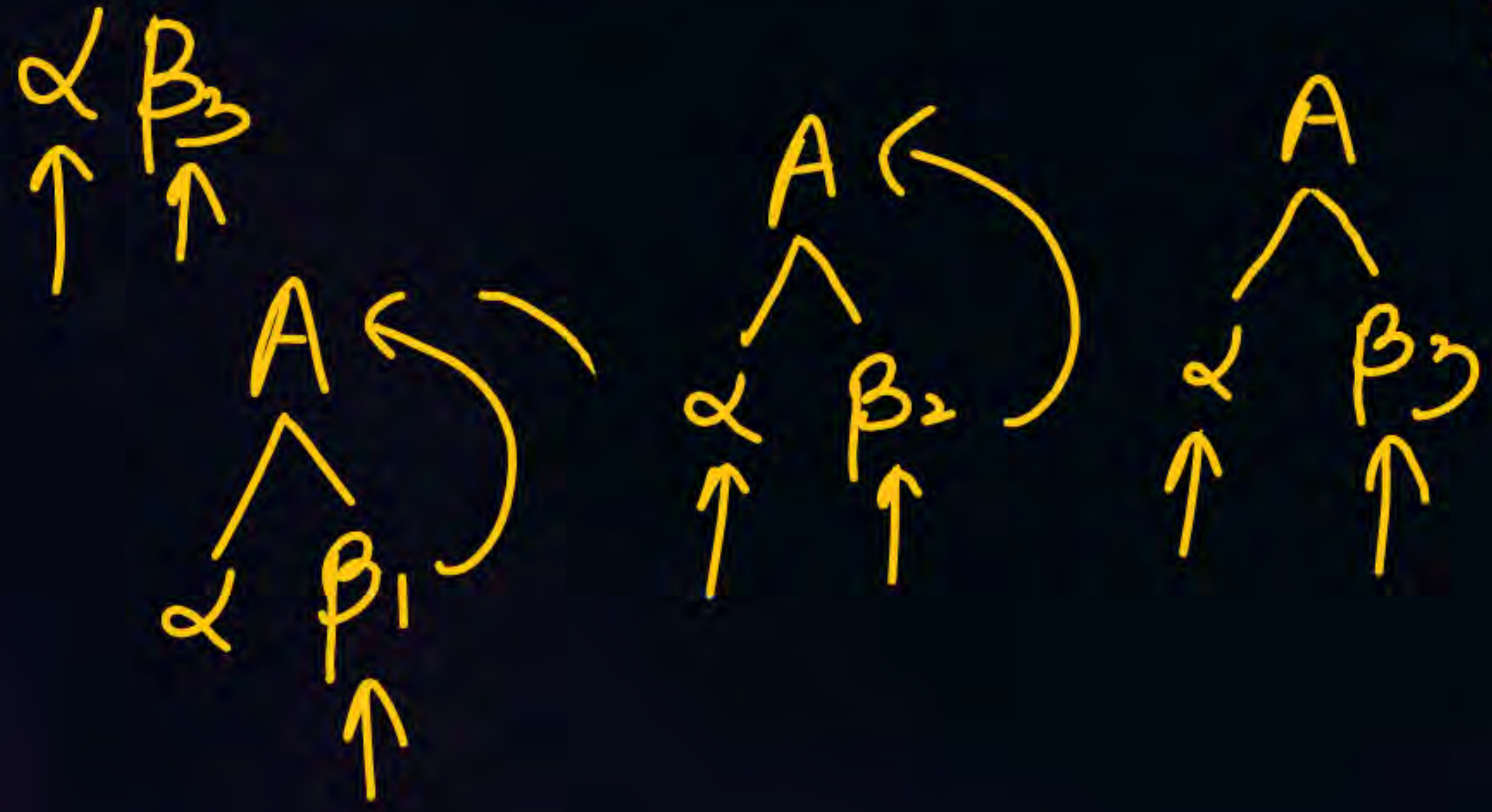
(3 min break)

Non deterministic grammar leads to back tracking

$$A \rightarrow \alpha\beta_1 \mid \alpha\beta_2 \mid \alpha\beta_3$$



Common prefix



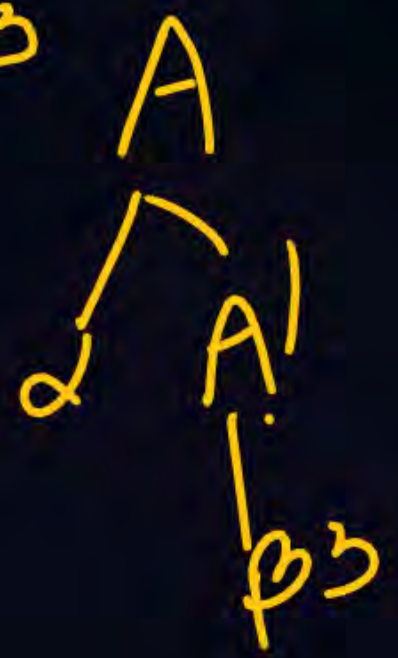
$A \rightarrow \alpha\beta_1 / \alpha\beta_2 / \alpha\beta_3$

left factoring

$A \rightarrow \alpha A'$

$A' \rightarrow \beta_1 / \beta_2 / \beta_3$

$\alpha\beta_3$



TDP

LR ✗

NDG ✗

---

1) eliminate LR

2) left factoring

$$S \rightarrow iEtS \mid iEtSeS \mid a$$
$$E \rightarrow b$$

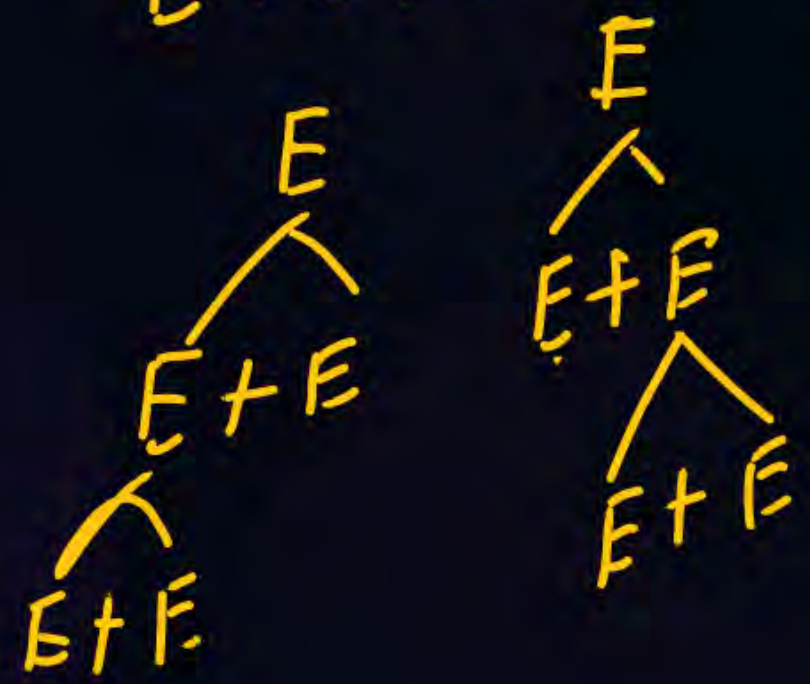
→ ambiguous.

Sentential form intermediate string

$$E \rightarrow E+E \mid E*E \mid id$$

$$id + id * id$$

$$E+E+E$$



$$\left.\begin{aligned} E &\Rightarrow E+E \\ &\Rightarrow E+E*E \\ &\Rightarrow id+E*E \end{aligned}\right\} \text{Sentential form}$$
$$\Rightarrow id+id*E$$
$$\Rightarrow id+id*id \rightarrow \frac{\text{string}}{\text{word}}$$

$$S \rightarrow iEtS \mid iEtSeS \mid a$$
$$E \rightarrow b$$

→ ambiguous.

Sentential form intermediate string

$$E \rightarrow E+E \mid E*E \mid id$$

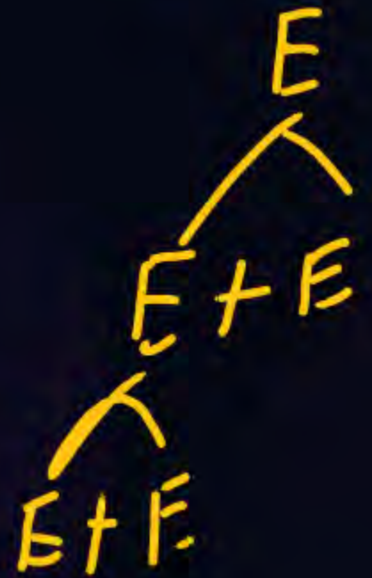
$id + id * id$

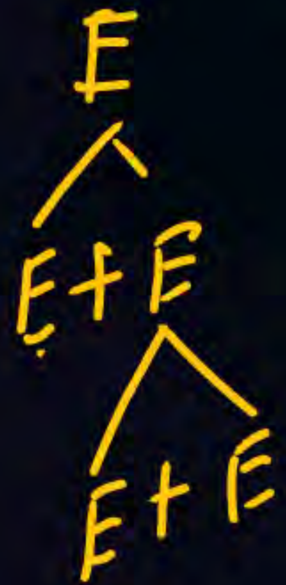
$$\begin{aligned} E &\Rightarrow E+E \\ &\Rightarrow E+E*E \\ &\Rightarrow id+E*E \\ &\Rightarrow id+id*E \\ &\Rightarrow id+id*id \end{aligned}$$

Sentential form

$id+id*id$ → string / word

$E+E+E$





$$S \to iEtS \mid iEtSeS \mid a$$
$$E \to b$$

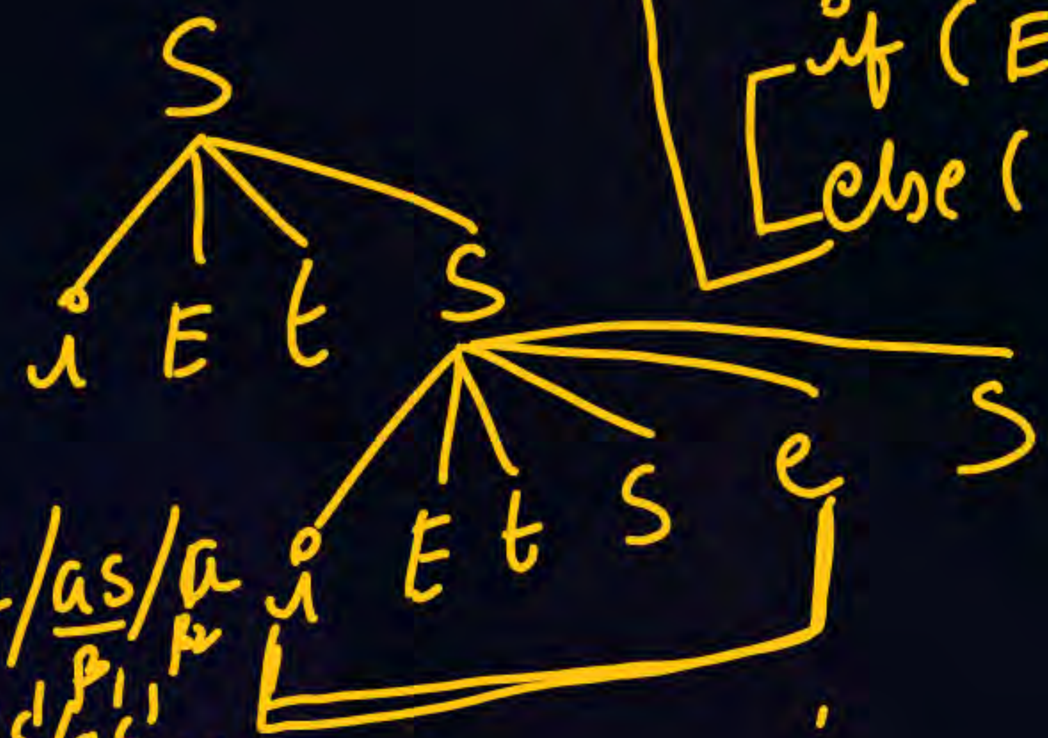
→ ambiguous.

Sentential form intermediate string

iEtiEtSeS

if (E) S
if (E) S
else ( )

① ✓



②

5 min

$S \to Sa \mid as \mid a$

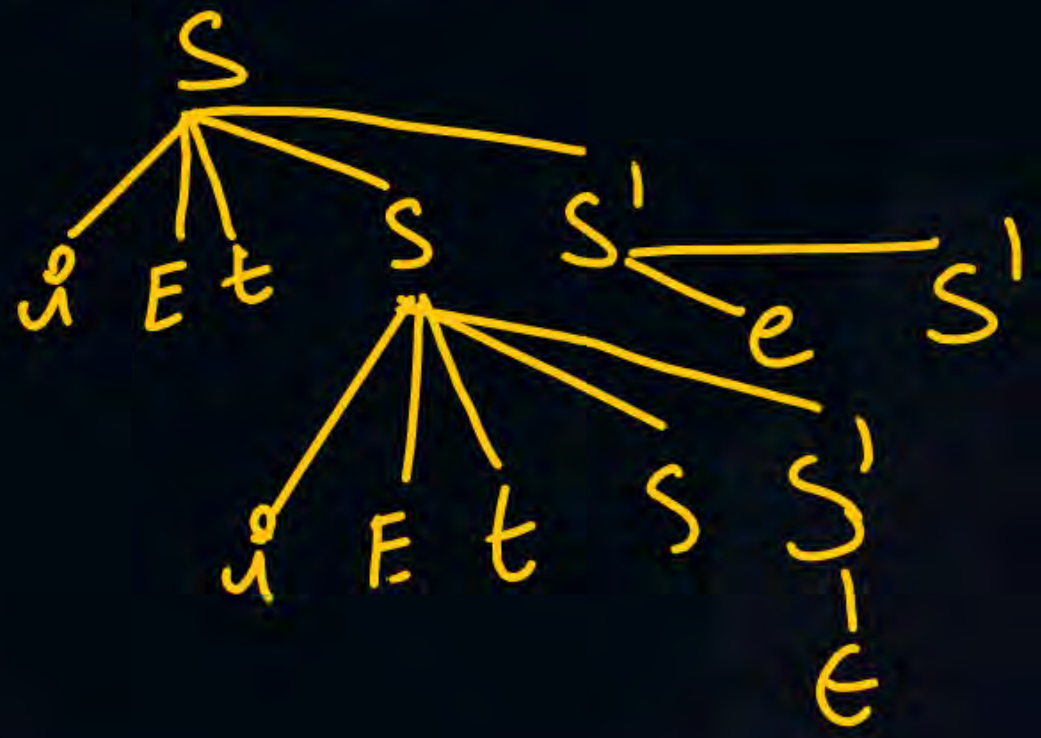
$S \to asS' \mid aS'$

$S' \to aS \mid \epsilon$

$S \to \underline{iEtS}\epsilon$
$\quad / \underline{iEtS}eS$
$\quad / a$
$E \to b$

$\xrightarrow{LF}$

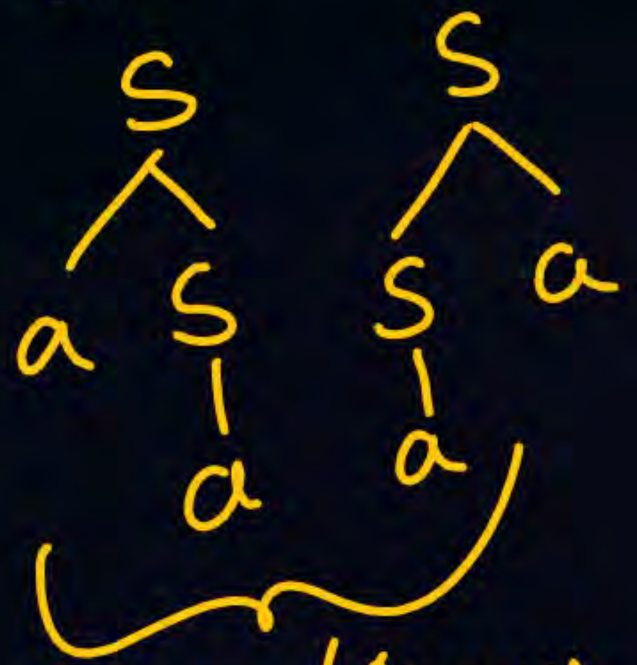
$S \to iEtSS'$
$\quad / a$
$S' \to \epsilon / eS$
$E \to b$



eliminating non determinism may not eliminate ambiguity

$$\underset{A}{\underline{S}} \to \underset{A\alpha}{\underline{S}a} \;/\; \underset{\beta_1}{\underline{aS}} \;/\; \underset{\beta_2}{\underline{a}}$$

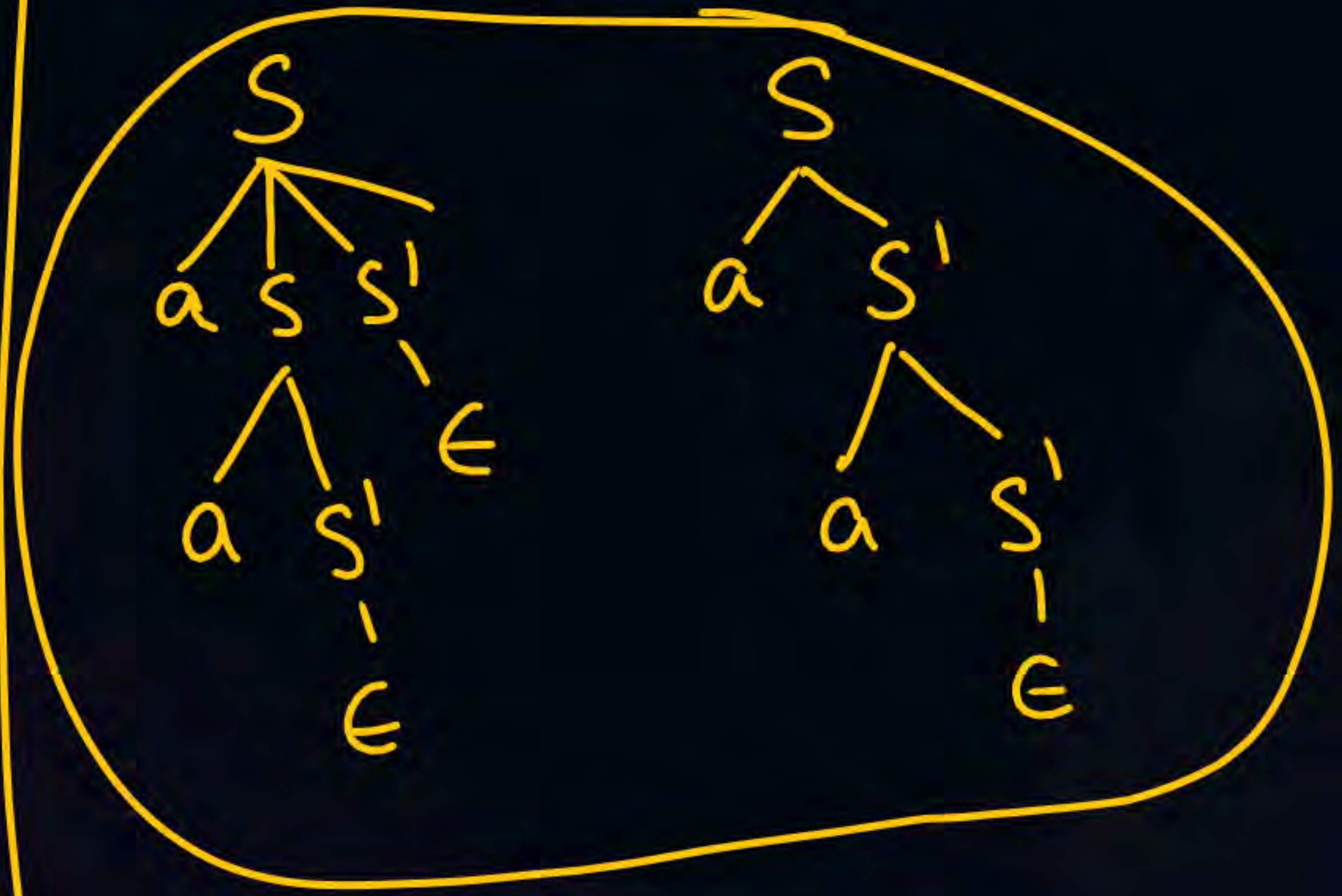
aa



ambiguous

$$S \to aSS' \;/\; aS'$$

$$S' \to aS' \;/\; \epsilon$$



Elimination left recursion may not eliminate ambiguity

$S \rightarrow aSSbS$
$\quad / aSaSb$
$\quad / abb$
$\quad / b$

$S \rightarrow aS' / b$
$S' \rightarrow SSbS / SaSb / bb / b$

$S' \rightarrow SS'' / bb / b$
$S'' \rightarrow SbS / aSb$

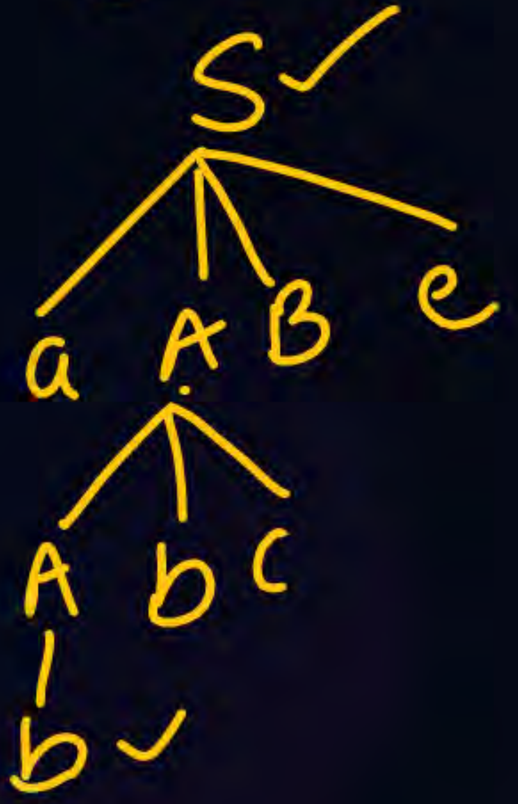
$S \rightarrow bSSaaS$
$\quad / bSSaSb$
$\quad / bSb$
$\quad / a$

$S \rightarrow bSS' / a$
$S' \rightarrow SaaS / SaSb / b$

$S' \rightarrow SaS'' / b$
$S'' \rightarrow aS / Sb$

(SA) Parsers (× Ambiguous)

Top down parser
(×LR, ×ND)



$S \rightarrow aABe$
$A \rightarrow \underline{Abc} / b$
$B \rightarrow \underline{d}$

$\underline{abbcde}$

handle → RHS of any production

Bottom up parser

The main purpose of a parser is, given a grammar and a string, we have to find out whether the string is generated by the grammar

The main decision of TDP is which production to use

$S \rightarrow \alpha_1 / \alpha_2 / \alpha_3$

The main decision of BUP is to know when to reduction

$S \to aABe$
$A \to Abc / b$
$B \to d$

TDP uses LMD ✓

$S \Rightarrow aABe$
$\Rightarrow aAbcBe$
$\Rightarrow abbcBe$
$\Rightarrow abbcde$

3 sequential forms

BUP uses reverse of RMD ✓

$S \Rightarrow aABe$
$\Rightarrow aAde$
$\Rightarrow aAbcde$
$\Rightarrow abbcde$

aABe
d
Abc
b

4 handles

Top down parsers
(TDP)

TDP with
full Backtracking

guntur ← 2004 → +2 → c prog

I bench July - 21 2004

BTech-ECE

2004 → MGIT → Btech

Vasavi →

Gate CS

GNN

Gate

134

IISc

2007 Feb 14

msc engg

2005 → auto →

2006 → Nov 14

Top down parsers

- TDP with full backtracking
  - Brute force method
- TDP without backtracking
  - Recursive descent
  - Non Recursive descent LL(1)

THANK - YOU